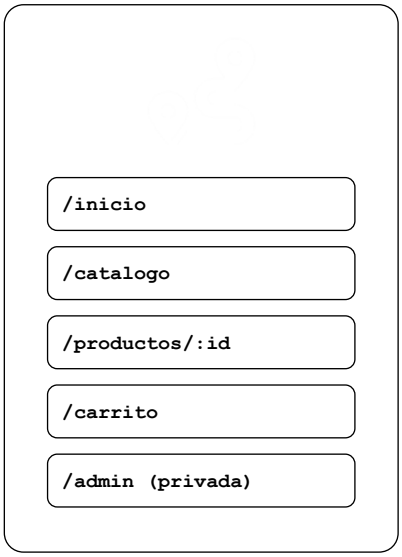
DESARROLLO WEB INTEGRADO · SEMANA 12 · SESIÓN 12

Sistema de rutas de Angular y comunicación entre componentes

Unidad 3 · Integración de proyectos web (back-end y front-end)

Mag. Ing. Marco Antonio Flores Torres
Universidad Tecnológica del Perú · Lima Sur



1

01

Inicio

Rompe hielo, repaso de la sesión 11 y saberes previos

8 minutos

2

1 · INICIO · ROMPE HIELO

Rompe hielo: «Adivina la URL»

En equipos, escriban la URL que creen que tiene cada pantalla. Ganan los que identifiquen qué parte de la URL cambia.

- **Detalle de una laptop en una tienda en línea**
  tienda.pe/productos/??
- **Restaurantes filtrados por «pollo a la brasa»**
  app.pe/restaurantes?????
- **Episodio 3 de la temporada 2 de una serie**
  streaming.com/serie/??/??
- **Panel de notas que solo ve el docente**
  campus.pe/???? (privada)

Desarrollo Web Integrado · Sesión 12 · UTP 3

3

1 · INICIO · REPASO SESIÓN 11

Repaso rápido: tres preguntas sencillas

1	¿Qué es un componente en Angular y qué tres partes lo forman?
2	¿Qué comando del Angular CLI usamos para crear un componente nuevo?
3	¿Con qué sintaxis mostramos el valor de una variable dentro de la plantilla HTML?

Desarrollo Web Integrado · Sesión 12 · UTP 4

4

1 · INICIO · CONTEXTUALIZACIÓN

¿Por qué este tema es importante para ti?

Tu proyecto integrador

El avance APF3 (semana 15) exige un front-end Angular con varias pantallas conectado a tu API Spring Boot.

Empleabilidad

Router y comunicación entre componentes aparecen en casi todas las ofertas de desarrollador front-end o full stack con Angular.

Continuidad con la unidad 2

Los roles y el JWT que implementaste en Spring Security se reflejan en el front-end con rutas protegidas.

Hoy, TechStore Web pasa de una sola pantalla a una aplicación completa

5

1 · INICIO · SABERES PREVIOS

¿Qué sabemos ya?

Piensa en una app que usas todos los días: Yape, Rappi, el campus virtual o una tienda en línea.

Pregunta 1

Cuando cambias de pantalla en una app web moderna, ¿la página completa se vuelve a descargar o solo cambia una parte?

Pregunta 2

Agregas un producto al carrito en una pantalla. ¿Cómo «se entera» el ícono del carrito de la cabecera?

Pregunta 3

En Spring Boot escribiste @GetMapping("/productos/{id}"). ¿Qué crees que hace Angular con /productos/15?

No hay respuestas incorrectas: anótalas y las comprobaremos durante la Transformación.

6

02

Utilidad

Preguntas retadoras, logro de la sesión y aplicaciones reales

7 minutos

7

2 · UTILIDAD · PREGUNTAS RETADORAS

Tres preguntas retadoras sobre la sesión 11

1	Un ProductCard declara producto = input.required<Producto>(). El padre escribe <app-product-card /> sin pasar nada. ¿Qué ocurre?
2	En la clase tienes contador = signal(0). Un alumno escribe this.contador = 5 para cambiarlo. ¿Funciona? ¿Cómo debe hacerse?
3	Tienes @for (p of productos(); track p.id). ¿Por qué Angular exige track y por qué conviene usar el id y no el índice?

8

2 · UTILIDAD · RESPUESTAS

Respuestas y explicación

1. Error de compilación

Un input requerido obliga al padre a enviarlo: <app-product-card [producto]="p" />. Angular avisa el error al compilar, antes de ejecutar la app.

2. No funciona así

Reemplaza el signal por un número y Angular deja de detectar cambios. Se usa this.contador.set(5) o this.contador.update(v => v + 1).

3. Para identificar cada fila

track le dice a Angular qué elemento es cuál. Con el id, si la lista se reordena o filtra, solo actualiza lo que cambió; con el índice recrea filas sin necesidad.

Hoy usaremos input(), signals y @for para conectar pantallas y componentes.

9

2 · UTILIDAD · LOGRO DE LA SESIÓN

Logro de la sesión

Al finalizar la sesión, el estudiante implementa la navegación de una aplicación Angular mediante el sistema de rutas (parámetros, rutas protegidas y carga diferida) y comunica componentes con input(), output(), model() y servicios con signals, aplicándolo a un caso de comercio electrónico.

Conocimientos

Rutas, RouterOutlet, RouterLink, parámetros, guards, lazy loading, input, output, model y servicios.

Habilidades

Configurar app.routes.ts, navegar por código y compartir estado entre componentes sin duplicar datos.

Aplicación profesional

Construir el front-end del proyecto integrador con pantallas protegidas según el rol del usuario.

10

2 · UTILIDAD · IMPORTANCIA

¿Para qué sirve lo que aprenderás hoy?

URLs que se comparten

Cada pantalla tiene su dirección: el usuario guarda o envía el enlace de un producto.

Botón Atrás que funciona

El router usa el historial del navegador aunque la página nunca se recargue.

Aplicaciones más rápidas

Con carga diferida (lazy loading) solo se descarga el código de la pantalla que se visita.

Pantallas protegidas

Los guards impiden entrar a vistas privadas sin sesión o sin el rol correcto.

Datos que fluyen

input, output y servicios mantienen sincronizados el catálogo, el carrito y la cabecera.

Código mantenible

Componentes pequeños y reutilizables que se comunican con contratos claros.

Desarrollo Web Integrado · Sesión 12 · UTP 11

11

2 · UTILIDAD · EJEMPLOS ACTUALES

Ejemplos del mundo actual

Tienda en línea

Catálogo, /productos/:id, filtros con ?categoria= y un carrito que se ve en toda la app.

Delivery de comida

Lista de restaurantes, detalle del local y pedido en curso compartido entre pantallas.

Banca por internet

Todas las rutas protegidas: sin sesión válida, el guard te devuelve al inicio de sesión.

Campus virtual

Rutas hijas: /curso/:id/semana/:n. Vistas distintas para alumno y docente según el rol.

Desarrollo Web Integrado · Sesión 12 · UTP 12

12

03

Transformación

Sistema de rutas de Angular, comunicación entre componentes y ejemplos aplicados

25 minutos

13

3 · TRANSFORMACIÓN · MAPA DE TEMAS

Lo que aprenderemos en esta fase

A. Sistema de rutas

- SPA y navegación del lado del cliente
- Routes, provideRouter, RouterOutlet
- RouterLink y RouterLinkActive
- Parámetros de ruta y de consulta
- Navegación por código
- Rutas hijas y carga diferida
- Guards: rutas protegidas

B. Comunicación entre componentes

- Padre → hijo: input()
- Hijo → padre: output()
- Doble sentido: model()
- El padre usa al hijo: viewChild()
- Componentes lejanos: servicio + signals
- Proyección de contenido: ng-content

C. Ejemplos aplicados

- Ejemplo 1: catálogo con detalle por id
- Ejemplo 2: carrito compartido en TechStore
- Ejemplo 3: intranet de notas con roles
- Errores frecuentes y cómo resolverlos

14

3 · TRANSFORMACIÓN · A · CONCEPTO PREVIO

SPA: una sola página, muchas pantallas

Aplicación multipágina (tradicional)

Navegador	Servidor
→ clic en /catalogo	
	← HTML completo
→ clic en /producto	
	← HTML completo
→ clic en /carrito	
	← HTML completo

La página se recarga en cada clic.

SPA con Angular Router

index.html se descarga una sola vez

`<app-header /> (no cambia)`

<router-outlet />
aquí cambia la pantalla según la URL

Solo cambia la zona del router-outlet.

15

3 · TRANSFORMACIÓN · A · ¿CÓMO FUNCIONA?

¿Qué hace el Router cuando cambia la URL?

1 → 2 → 3 → 4 → 5

Cambia la URL

Clic en un enlace, botón Atrás o URL escrita: /productos/3

Busca en Routes

Recorre la lista de rutas en orden, de arriba hacia abajo

Coincidencia

La primera que coincide: productos/:id con id = '3'

Revisa los guards

Si la ruta está protegida, pregunta si puede entrar

Dibuja la vista

Muestra ProductoDetalle dentro de <router-outlet />

Regla de oro: el Router usa la PRIMERA ruta que coincide. Por eso el orden de la lista importa.

16

3 · TRANSFORMACIÓN · A · PIEZAS DEL ROUTER

Las piezas del sistema de rutas

Pieza	¿Qué es?	¿Dónde se usa?
Routes	Lista (arreglo) de rutas: cada una une un path con un componente	app.routes.ts
provideRouter()	Activa el Router en la aplicación con la lista de rutas	app.config.ts
<router-outlet />	Lugar de la pantalla donde se dibuja el componente de la ruta	app.html (o un layout)
routerLink	Directiva para navegar desde la plantilla sin recargar	Enlaces y botones del HTML
routerLinkActive	Agrega una clase CSS al enlace de la ruta activa	Menús de navegación
Router	Servicio para navegar desde el código TypeScript	Métodos: navigate(), navigateByUrl()
ActivatedRoute	Información de la ruta actual (parámetros, datos)	Alternativa a recibir parámetros con input()
CanActivateFn	Función guard que permite o bloquea la entrada a una ruta	Propiedad canActivate de una ruta

17

3 · TRANSFORMACIÓN · A · PASO 1

Paso 1: definir las rutas en app.routes.ts

src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { Inicio } from './pages/inicio/inicio';
import { Catalogo } from './pages/catalogo/catalogo';
import { NoEncontrado }
  from './pages/no-encontrado/no-encontrado';

export const routes: Routes = [
  { path: '', redirectTo: 'inicio',
    pathMatch: 'full' },
  { path: 'inicio', component: Inicio,
    title: 'TechStore | Inicio' },
  { path: 'catalogo', component: Catalogo,
    title: 'TechStore | Catálogo' },
  { path: '**', component: NoEncontrado },
];
```

- **path:** parte de la URL, sin barra inicial.
- **component:** componente que se mostrará.
- **redirectTo:** envía a otra ruta (aquí, de vacío a inicio).
- **pathMatch: 'full':** redirige solo si la URL está completamente vacía.
- **title:** texto de la pestaña del navegador.
- **'**':** comodín para la página 404; va al final.

18

3 · TRANSFORMACIÓN · A · PASO 2

Paso 2: activar el Router y ubicar el outlet

src/app/app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding }
  from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
  ],
};
```

src/app/app.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header }
  from './components/header/header';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header],
  templateUrl: './app.html',
  styleUrl: './app.scss',
})
export class App {}
```

src/app/app.html

```
<app-header />
<main class="contenedor">
  <router-outlet />   <!-- aquí cambia la vista -->
</main>
```

Ojo

Si olvidas RouterOutlet en imports, aparece el error NG8001.

19

3 · TRANSFORMACIÓN · A · NAVEGACIÓN

RouterLink y RouterLinkActive

components/header/header.html

```
<nav class="menu">
  <a routerLink="/inicio" routerLinkActive="activo"
     [routerLinkActiveOptions]="{ exact: true }">Inicio</a>
  <a routerLink="/catalogo"
     routerLinkActive="activo">Catálogo</a>
  <a [routerLink]="['/productos', 3]">Laptop Lenovo</a>
</nav>
```

components/header/header.ts (fragmento)

```
import { RouterLink, RouterLinkActive }
  from '@angular/router';
@Component({ selector: 'app-header',
  imports: [RouterLink, RouterLinkActive], ...})
```

routerLink

Navega sin recargar. Acepta texto ('/catalogo') o un arreglo de segmentos (['/productos', 3]).

routerLinkActive

Agrega la clase 'activo' al enlace de la ruta actual para resaltarlo con SASS.

20

3 · TRANSFORMACIÓN · A · ORDEN Y COMODINES

Redirecciones, comodín y orden de las rutas

Rutas configuradas (en este orden)

```
[
  { path: '', redirectTo: 'inicio',
    pathMatch: 'full' },
  { path: 'inicio', component: Inicio },
  { path: 'catalogo', component: Catalogo },
  { path: 'productos/:id',
    component: ProductoDetalle },
  { path: '**', component: NoEncontrado },
]
```

URL visitada	¿Qué se muestra?
/	Redirige a /inicio → Inicio
/catalogo	Catalogo
/productos/7	ProductoDetalle con id = '7'
/productos	NoEncontrado (falta el :id)
/ofertas	NoEncontrado (comodín **)

Error frecuente

Si pones { path: '**' } al inicio de la lista, TODAS las URL mostrarán la página 404, porque el comodín coincide con todo.

21

3 · TRANSFORMACIÓN · A · PARÁMETROS DE RUTA

Parámetros de ruta: /productos/:id

app.routes.ts

```
{ path: 'productos/:id', component: ProductoDetalle }
```

pages/producto-detalle/producto-detalle.ts

```
export class ProductoDetalle {
  private productos = inject(ProductoService);

  // Mismo nombre que :id → el Router lo llena
  id = input.required<string>();

  producto = computed(() =>
    this.productos.buscarPorId(Number(this.id())));
}
```

Claves

- **El nombre importa:** :id ↔ id = input().
- **Siempre es texto:** conviértelo con Number().

Alternativa clásica

```
route = inject(ActivatedRoute);
id = this.route.snapshot
       .paramMap.get('id');
```


22

3 · TRANSFORMACIÓN · A · PARÁMETROS DE CONSULTA

Parámetros de consulta: ?categoria=laptops

techstore.pe/catalogo?categoria=laptops&orden=precio

Enlace con query params

```
<a routerLink="/catalogo"
   [queryParams]="{ categoria: 'laptops' }">
   Laptops</a>
```

	Parámetro de ruta	Query param
Ejemplo	/productos/3	?categoria=laptops
¿Obligatorio?	Sí	No
Uso típico	Identificar un recurso	Filtrar, buscar, ordenar

pages/catalogo/catalogo.ts

```
export class Catalogo {
  private productos = inject(ProductoService);
  categoria = input<string>(); // opcional

  filtrados = computed(() => {
    const c = this.categoria();
    const todos = this.productos.lista();
    return c ? todos.filter(
      p => p.categoria === c) : todos;
  });
}
```


23

3 · TRANSFORMACIÓN · A · NAVEGACIÓN POR CÓDIGO

Navegación programática con el servicio Router

Dentro de cualquier componente

```
export class ProductoDetalle {
  private router = inject(Router);

  // Arreglo de segmentos → /productos/5
  verOtro(id: number) {
    this.router.navigate(['/productos', id]);
  }

  // Con query params → /catalogo?q=mouse
  buscar(texto: string) {
    this.router.navigate(['/catalogo'],
      { queryParams: { q: texto } });
  }

  // URL completa
  volver() { this.router.navigateByUrl('/catalogo'); }
}
```

navigate([...])

Arma la URL a partir de segmentos y admite opciones como queryParams.

navigateByUrl('...')

Recibe la URL completa como texto. Útil para volver a una dirección guardada.

24

3 · TRANSFORMACIÓN · A · RUTAS HIJAS

Rutas hijas (children) y layouts anidados

app.routes.ts

```
{
  path: 'docente',
  component: DocenteLayout,
  children: [
    { path: '', redirectTo: 'cursos',
      pathMatch: 'full' },
    { path: 'cursos', component: MisCursos },
    { path: 'notas/:cursoId',
      component: RegistroNotas },
  ],
}
```

/docente/notas/12

App → <router-outlet /> principal

DocenteLayout

Menú lateral

<router-outlet /> hijo

RegistroNotas cursoId = '12'

25

3 · TRANSFORMACIÓN · A · CARGA DIFERIDA

Lazy loading: cargar cada pantalla cuando se necesita

app.routes.ts

```
// Un componente
{ path: 'admin',
  loadComponent: () => import('./pages/admin/admin')
    .then(m => m.Admin) },

// Un grupo de rutas
{ path: 'docente',
  loadChildren: () => import('./docente/docente.routes')
    .then(m => m.DOCENTE_ROUTES) },
```

Sin lazy loading

main.js: inicio + catálogo + carrito + admin + docente

Con lazy loading

main.js (lo esencial) | admin.js | docente.js

Beneficio

La primera pantalla carga más rápido; el resto se descarga bajo demanda.

Atajo

Si la clase se exporta con export default, basta: loadComponent: () => import('./pages/admin/admin')

26

3 · TRANSFORMACIÓN · A · GUARDS

Guards: proteger rutas

guards/auth-guard.ts

```
export const authGuard: CanActivateFn =
  (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.logueado()) return true;    // pasa
  return router.createUrlTree(['/login'], // redirige
    { queryParams: { volver: state.url } });
};
```

Uso en app.routes.ts

```
{ path: 'admin', canActivate: [authGuard],
  loadComponent: () => import('./pages/admin/admin')... }
```

Usuario va a /admin

↓

authGuard: ¿logueado()?

Sí → true
muestra Admin

No → /login
?volver=/admin

Recuerda

El guard protege la navegación. La seguridad real de los datos está en Spring Security.

27

3 · TRANSFORMACIÓN · A · RESUMEN DE PROPIEDADES

Propiedades de una ruta: resumen

Propiedad	Para qué sirve	Ejemplo
path	Parte de la URL	'productos/:id'
component	Componente a mostrar (carga inmediata)	Catalogo
loadComponent	Componente con carga diferida	() => import('./pages/admin/admin')
loadChildren	Grupo de rutas con carga diferida	() => import('./docente/docente.routes')
children	Rutas hijas dentro de un layout	[{ path: 'cursos', ... }]
redirectTo + pathMatch	Redirigir a otra ruta	redirectTo: 'inicio', pathMatch: 'full'
canActivate	Guards que deciden si se entra	[authGuard]
title	Título de la pestaña del navegador	'TechStore | Carrito'
data	Datos fijos para la ruta	{ rol: 'DOCENTE' }

28

3 · TRANSFORMACIÓN · A · PUENTE CON SPRING BOOT

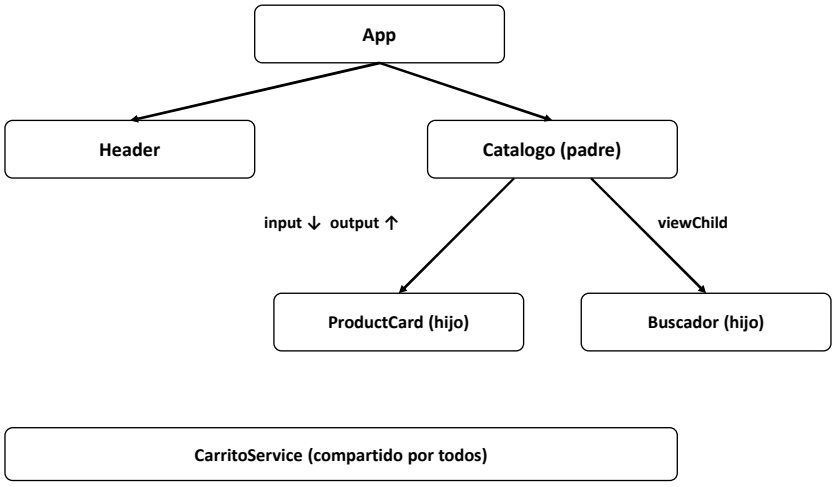
Lo que ya sabes en Spring Boot, ahora en Angular

Concepto	Spring Boot (back-end)	Angular (front-end)
Definir una ruta	@GetMapping("/productos")	{ path: 'productos', component: ... }
Parámetro en la URL	/{id} con @PathVariable Long id	:id con id = input.required<string>()
Parámetro de consulta	@RequestParam String categoria	?categoria= con categoria = input<string>()
Proteger un recurso	Spring Security, roles y JWT	Guard CanActivateFn
Objeto compartido	@Service (singleton) e inyección	@Injectable({ providedIn: 'root' }) e inject()
Recurso inexistente	Respuesta 404 Not Found	Ruta comodín '**'

29

3 · TRANSFORMACIÓN · B · COMUNICACIÓN

¿Cómo se comunican los componentes?



`input()`
Padre → hijo: le envía datos.

`output()`
Hijo → padre: avisa un evento.

`model()`
Doble sentido entre padre e hijo.

`viewChild()`
El padre usa métodos del hijo.

`Servicio + signals`
Componentes lejanos o hermanos.

30

3 · TRANSFORMACIÓN · B · PADRE → HIJO

input(): el padre envía datos al hijo

Hijo: components/product-card/product-card.ts

```
export class ProductCard {
  producto = input.required<Producto>();
  destacado = input(false); // por defecto
}
```

Hijo: product-card.html

```
<h3>{{ producto().nombre }}</h3>
@if (destacado()) { <span>¡Últimas unidades!</span> }
```

Padre: catalogo.html

```
@for (p of filtrados(); track p.id) {
  <app-product-card [producto]="p"
     [destacado]="p.stock < 5" />
}
```

Recuerda

- **[producto]="p":** corchetes envían el valor.
- **producto():** se lee como signal.
- **Solo lectura:** el hijo no lo modifica.

31

3 · TRANSFORMACIÓN · B · HIJO → PADRE

output(): el hijo avisa al padre

Hijo: product-card.ts

```
export class ProductCard {
  producto = input.required<Producto>();
  agregar = output<Producto>();

  onAgregar() {
    this.agregar.emit(this.producto());
  }
}
```

Hijo: product-card.html

```
<button (click)="onAgregar()">
  Agregar al carrito</button>
```

Padre: catalogo.html y catalogo.ts

```
<app-product-card [producto]="p"
   (agregar)="alAgregar($event)" />

alAgregar(p: Producto) {
  this.carrito.agregar(p);
}
```

Flujo

clic → emit(producto) → (agregar) → alAgregar($event)

32

3 · TRANSFORMACIÓN · B · DOBLE SENTIDO

model(): enlace en ambos sentidos

Hijo: selector-cantidad.ts

```
export class SelectorCantidad {
  cantidad = model(1);
  max = input(10);

  sumar() { if (this.cantidad() < this.max())
    this.cantidad.update(c => c + 1); }
  restar() { if (this.cantidad() > 1)
    this.cantidad.update(c => c - 1); }
}
```

Hijo: selector-cantidad.html

```
<button (click)="restar()">-</button>
<span>{{ cantidad() }}</span>
<button (click)="sumar()">+</button>
```

Padre: producto-detalle

```
cantidad = signal(1);

<app-selector-cantidad
  [(cantidad)]="cantidad"
  [max]="p.stock" />
```

Banana en caja [( )]

Corchetes: el padre envía el valor. Paréntesis: el hijo devuelve los cambios. Ambos quedan sincronizados.

33

3 · TRANSFORMACIÓN · B · EL PADRE USA AL HIJO

viewChild(): el padre accede al hijo

Hijo: buscador.ts

```
export class Buscador {
  texto = signal('');
  limpiar() { this.texto.set(''); }
}
```

Padre: catalogo.ts

```
export class Catalogo {
  buscador = viewChild.required(Buscador);

  limpiarFiltros() {
    this.buscador().limpiar();
  }
}
```

Padre: catalogo.html

```
<app-buscador />
<button (click)="limpiarFiltros()">Limpiar</button>
```

Úsalo con moderación

Prefiere input y output: acoplan menos.

34

3 · TRANSFORMACIÓN · B · SERVICIO COMPARTIDO

Servicio con signals: comunicar componentes lejanos

services/carrito-service.ts

```
@Injectable({ providedIn: 'root' })
export class CarritoService {
  private readonly _items = signal<ItemCarrito[]>([]);
  readonly items = this._items.asReadonly();

  readonly cantidadTotal = computed(() =>
    this._items().reduce((s, i) => s + i.cantidad, 0));
  readonly total = computed(() => this._items()
    .reduce((s, i) => s + i.producto.precio * i.cantidad, 0));

  agregar(producto: Producto, cantidad = 1) {
    this._items.update(items => {
      const existe = items.find(i => i.producto.id === producto.id);
      return existe
        ? items.map(i => i.producto.id === producto.id
            ? { ...i, cantidad: i.cantidad + cantidad } : i)
        : [...items, { producto, cantidad }];
    });
  }
}
```

header.ts / header.html

```
carrito = inject(CarritoService);

Carrito
({{ carrito.cantidadTotal() }})
```

Una sola fuente

Todos inyectan la misma instancia. Si el catálogo agrega, la cabecera se actualiza sola.

35

3 · TRANSFORMACIÓN · B · PROYECCIÓN DE CONTENIDO

ng-content: el padre envía HTML al hijo

Hijo: panel.ts

```
@Component({
  selector: 'app-panel',
  template: `<section class="panel">
    <h3>{{ titulo() }}</h3>
    <ng-content />
  </section>`,
})
export class Panel { titulo = input(''); }
```

Padre: carrito.html

```
<app-panel titulo="Resumen">
  <p>Productos:
    {{ carrito.cantidadTotal() }}</p>
  <p>Total:
    {{ carrito.total() }}</p>
</app-panel>
```

Idea

El Panel pone el marco y el título; el padre decide qué contenido va dentro. Un mismo Panel sirve para el carrito, el perfil o el detalle.

36

3 · TRANSFORMACIÓN · B · ¿CUÁL ELIJO?

¿Qué mecanismo de comunicación elijo?

Situación	Mecanismo	Ejemplo en TechStore
El padre debe mostrar datos en el hijo	input()	Catalogo envía cada producto a ProductCard
El hijo debe avisar que algo pasó	output()	ProductCard avisa «agregar al carrito»
Un control que el padre y el hijo modifican	model()	SelectorCantidad en el detalle
El padre debe ejecutar algo del hijo	viewChild()	Catalogo limpia el Buscador
Componentes sin relación directa	Servicio + signal	Header y Carrito comparten CarritoService
El padre envía HTML al hijo	ng-content	Panel de resumen del carrito
Datos que vienen de la URL	Router + input()	ProductoDetalle recibe :id

37

3 · TRANSFORMACIÓN · B · ANTES Y AHORA

Si encuentras código antiguo: decoradores vs funciones

Propósito	Forma clásica (decoradores)	Forma moderna (Angular 17+)
Recibir datos	@Input() producto!: Producto;	producto = input.required<Producto>();
Emitir eventos	@Output() agregar = new EventEmitter<Producto>();	agregar = output<Producto>();
Doble sentido	@Input() + @Output() xChange	cantidad = model(1);
Acceder al hijo	@ViewChild(Buscador) buscador!: Buscador;	buscador = viewChild.required(Buscador);
Leer el valor	this.producto.nombre	this.producto().nombre

Ambas formas conviven, pero no las mezcles en un mismo componente: elige la moderna para proyectos nuevos.

38

3 · TRANSFORMACIÓN · C · EJEMPLO 1

Ejemplo 1: catálogo con detalle por producto

1. product-card.html

```
<a [routerLink]="['/productos', producto().id]">
  Ver detalle</a>
```

2. producto-detalle.ts

```
export class ProductoDetalle {
  private productos = inject(ProductoService);
  id = input.required<string>();
  producto = computed(() =>
    this.productos.buscarPorId(
      Number(this.id())));
}
```

3. producto-detalle.html

```
@if (producto(); as p) {
  <h2>{{ p.nombre }}</h2>
  <p>{{ p.precio | currency:'PEN':'S/ ' }}</p>
  <p>Stock: {{ p.stock }}</p>
} @else {
  <p>Producto no encontrado</p>
  <a routerLink="/catalogo">Volver</a>
}
```

Prueba: /productos/2 y /productos/99

39

3 · TRANSFORMACIÓN · C · EJEMPLO 2

Ejemplo 2: carrito compartido en TechStore

ProductCard	Catalogo	CarritoService	Header
clic en «Agregar» agregar.emit(p)	(agregar)= "alAgregar($event)"	_items.update(...) cantidadTotal() cambia	muestra Carrito (3)

catalogo.ts

```
export class Catalogo {
  private carrito = inject(CarritoService);
  alAgregar(p: Producto) {
    this.carrito.agregar(p);
  }
}
```

header.html

```
<a routerLink="/carrito"
   routerLinkActive="activo">
  Carrito
  ({{ carrito.cantidadTotal() }})
</a>
```


40

3 · TRANSFORMACIÓN · C · EJEMPLO 3

Ejemplo 3: intranet de notas con roles

app.routes.ts

```
{
  path: 'docente',
  canActivate: [rolGuard],
  data: { rol: 'DOCENTE' },
  loadChildren: () =>
    import('./docente/docente.routes')
      .then(m => m.DOCENTE_ROUTES),
},
{ path: 'login', component: Login },
```

guards/rol-guard.ts

```
export const rolGuard: CanActivateFn =
  (route) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.rol() === route.data['rol']
    ? true
    : router.createUrlTree(['/login']);
};
```

Resultado

Un ALUMNO que escribe /docente es enviado a /login.

41

3 · TRANSFORMACIÓN · C · ERRORES FRECUENTES

Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
NG8001: 'router-outlet' is not a known element	No se importó RouterOutlet	Agregar RouterOutlet en imports del componente
El enlace recarga toda la página	Se usó href en vez de routerLink	Usar routerLink e importar RouterLink
Todas las URL muestran la página 404	La ruta '**' no está al final	Mover el comodín al final de la lista
El input id llega undefined	Falta withComponentInputBinding() o el nombre no coincide	Agregarlo en provideRouter y usar el mismo nombre que :id
«Inicio» siempre aparece activo	Coincidencia parcial de routerLinkActive	Usar [routerLinkActiveOptions]="{ exact: true }"
El contador del carrito no cambia	Cada componente tiene su propia lista	Usar un servicio providedIn: 'root' con signal
404 al recargar /productos/3 en producción	El servidor busca un archivo que no existe	Redirigir todo a index.html (sesión 17)

42

04

Práctica

Tres ejercicios retadores: inicial, intermedio y avanzado

12 minutos

43

4 · PRÁCTICA · EJERCICIO 1 · INICIAL

Ejercicio 1 (inicial): menú de navegación de TechStore

Objetivo

Crear cuatro páginas y navegar entre ellas con un menú que resalte la opción activa.

Requisitos

- Genera con el CLI: inicio, nosotros, contacto y no-encontrado dentro de pages/.
- Configura app.routes.ts: '' redirige a inicio y '**' muestra no-encontrado.
- Cada ruta tiene un title: 'TechStore | Inicio', etc.
- Crea un Header con enlaces routerLink y la clase 'activo' con routerLinkActive.
- Estiliza .activo con una variable SASS de la sesión 11.

Usarás

- **Sesión 11:** ng g c, componentes standalone y SASS.
- **Hoy:** Routes, RouterOutlet, routerLink y routerLinkActive.

Resultado esperado

Al hacer clic, cambia la pantalla sin recargar, el enlace activo se resalta y /xyz muestra la página 404.

44

4 · PRÁCTICA · EJERCICIO 2 · INTERMEDIO

Ejercicio 2 (intermedio): detalle de producto por id

Objetivo

Mostrar la ficha de un producto según el id de la URL y volver al catálogo por código.

Requisitos

- Crea ProductoService con un signal de 4 productos y el método buscarPorId(id).
- Agrega la ruta productos/:id y activa withComponentInputBinding().
- En ProductCard, agrega un enlace [routerLink]="['/productos', producto().id]".
- En ProductoDetalle recibe id con input.required y obtén el producto con computed.
- Si el id no existe, muestra «Producto no encontrado».
- Botón «Volver» que use router.navigate(['/catalogo']).

Usarás

- **Sesión 11:** input(), @for, @if, signals y pipes.
- **Hoy:** parámetros de ruta, input binding y Router.

Resultado esperado

/productos/2 muestra el mouse; /productos/99 muestra el mensaje; «Volver» regresa al catálogo.

4 · PRÁCTICA · EJERCICIO 3 · AVANZADO

Ejercicio 3 (avanzado): favoritos protegidos

Objetivo

Marcar productos como favoritos desde el catálogo y verlos en una página protegida y de carga diferida.

Requisitos

- FavoritosService con signal de ids, alternar(id), esFavorito(id) y total (computed).
- ProductCard emite favorito con output<number>() y recibe esFavorito con input().
- El Header muestra «Favoritos (n)» leyendo el servicio.
- AuthService con logueado = signal(false), ingresar() y salir().
- Ruta /favoritos con loadComponent y canActivate: [authGuard].
- Sin sesión, el guard redirige a /login?volver=/favoritos.

Usarás

- **Sesiones 8 y 9:** idea de roles y sesión.
- **Hoy:** output, servicio con signals, guard y lazy loading.

Resultado esperado

El contador del header cambia al marcar; sin sesión /favoritos redirige a /login; con sesión lista los favoritos.

4 · PRÁCTICA · SOLUCIÓN 1

Solución del ejercicio 1

Paso 1 · Terminal

```
ng g c pages/inicio
ng g c pages/nosotros
ng g c pages/contacto
ng g c pages/no-encontrado
```

Paso 2 · app.routes.ts

```
export const routes: Routes = [
  { path: '', redirectTo: 'inicio',
    pathMatch: 'full' },
  { path: 'inicio', component: Inicio,
    title: 'TechStore | Inicio' },
  { path: 'nosotros', component: Nosotros,
    title: 'TechStore | Nosotros' },
  { path: 'contacto', component: Contacto,
    title: 'TechStore | Contacto' },
  { path: '**', component: NoEncontrado },
];
```

Paso 3 · header.html

```
<nav class="menu">
  <a routerLink="/inicio" routerLinkActive="activo"
     [routerLinkActiveOptions]="{ exact: true }">
     Inicio</a>
  <a routerLink="/nosotros"
     routerLinkActive="activo">Nosotros</a>
  <a routerLink="/contacto"
     routerLinkActive="activo">Contacto</a>
</nav>
```

Paso 4 · header.scss

```
@use '../../../scss/variables' as v;
// _variables.scss: $rojo-utp: #E30613;
.menu a { margin-right: 16px; }
.menu a.activo { color: v.$rojo-utp;
  border-bottom: 2px solid v.$rojo-utp; }
```


4 · PRÁCTICA · SOLUCIÓN 2 · PARTE 1

Solución del ejercicio 2 (1/2): servicio y rutas

services/producto-service.ts

```
@Injectable({ providedIn: 'root' })
export class ProductoService {
  private readonly _lista = signal<Producto[]>([
    { id: 1, nombre: 'Laptop Lenovo', precio: 2499,
      stock: 8, categoria: 'laptops' },
    { id: 2, nombre: 'Mouse Logitech', precio: 49.9,
      stock: 25, categoria: 'accesorios' },
    { id: 3, nombre: 'Teclado Redragon', precio: 159,
      stock: 0, categoria: 'accesorios' },
    { id: 4, nombre: 'Monitor Samsung', precio: 599,
      stock: 4, categoria: 'monitores' },
  ]);
  readonly lista = this._lista.asReadonly();

  buscarPorId(id: number) {
    return this._lista().find(p => p.id === id);
  }
}
```

app.routes.ts (agregar)

```
{ path: 'productos/:id',
  component: ProductoDetalle,
  title: 'TechStore | Detalle' },
```

app.config.ts

```
provideRouter(routes,
  withComponentInputBinding())
```

product-card.html

```
<a [routerLink]="['/productos',
   producto().id]">Ver detalle</a>
```

4 · PRÁCTICA · SOLUCIÓN 2 · PARTE 2

Solución del ejercicio 2 (2/2): componente de detalle

pages/producto-detalle/producto-detalle.ts

```
@Component({
  selector: 'app-producto-detalle',
  imports: [CurrencyPipe],
  templateUrl: './producto-detalle.html',
  styleUrl: './producto-detalle.scss',
})
export class ProductoDetalle {
  private productos = inject(ProductoService);
  private router = inject(Router);

  id = input.required<string>();
  producto = computed(() =>
    this.productos.buscarPorId(Number(this.id())));

  volver() { this.router.navigate(['/catalogo']); }
}
```

producto-detalle.html

```
@if (producto(); as p) {
  <h2>{{ p.nombre }}</h2>
  <p>{{ p.precio | currency:'PEN':'S/ ' }}</p>
  <p>Stock: {{ p.stock }}</p>
} @else {
  <p>Producto no encontrado</p>
}
<button (click)="volver()">Volver</button>
```

Verifica

/productos/2 · /productos/99 · Volver

4 · PRÁCTICA · SOLUCIÓN 3 · PARTE 1

Solución del ejercicio 3 (1/2): servicios y tarjeta

services/favoritos-service.ts

```
@Injectable({ providedIn: 'root' })
export class FavoritosService {
  private readonly _ids = signal<number[]>([]);
  readonly total = computed(() => this._ids().length);

  esFavorito(id: number) {
    return this._ids().includes(id); }
  alternar(id: number) {
    this._ids.update(ids => ids.includes(id)
      ? ids.filter(x => x !== id) : [...ids, id]); }
}
```

services/auth-service.ts

```
@Injectable({ providedIn: 'root' })
export class AuthService { logueado = signal(false);
  ingresar() { this.logueado.set(true); }
  salir() { this.logueado.set(false); } }
```

product-card.ts y .html

```
esFavorito = input(false);
favorito = output<number>();

<button [class.on]="esFavorito()"
  (click)="favorito.emit(producto().id)">
  Favorito</button>
```

catalogo.html

```
<app-product-card [producto]="p"
  [esFavorito]="favs.esFavorito(p.id)"
  (favorito)="favs.alternar($event)" />
```

4 · PRÁCTICA · SOLUCIÓN 3 · PARTE 2

Solución del ejercicio 3 (2/2): ruta protegida y diferida

app.routes.ts

```
{ path: 'login', component: Login },
{ path: 'favoritos', canActivate: [authGuard],
  loadComponent: () =>
import('./pages/favoritos/favoritos')
    .then(m => m.Favoritos) },
```

pages/login/login.ts

```
export class Login {
  private auth = inject(AuthService);
  private router = inject(Router);
  volver = input<string>(); // ?volver=...
  entrar() {
    this.auth.ingresar();
    this.router.navigateByUrl(
      this.volver() ?? '/inicio');
  }
}
```

pages/favoritos/favoritos.ts

```
export class Favoritos {
  private productos = inject(ProductoService);
  private favs = inject(FavoritosService);
  lista = computed(() => this.productos
    .lista().filter(p =>
      this.favs.esFavorito(p.id)));
}
```

Guard

Usa el authGuard visto en la Transformación (redirige a /login con ?volver=).

51

05

Cierre

Conclusiones construidas por ustedes, evaluación y mensaje final

8 minutos

52

5 · CIERRE · DINÁMICA

Dinámica: «El GPS de la sesión»

Cada equipo recibe una «parada». En 1 minuto escriban una conclusión y un ejemplo de TechStore. Luego recorremos el GPS en orden.

Parada 1	Parada 2	Parada 3	Parada 4	Parada 5
Rutas y outlet	**Parámetros**	**Guards y lazy**	**input / output / model**	**Servicios**
¿Qué hace el Router y dónde dibuja la pantalla?	¿Cuándo uso :id y cuándo ?categoria=?	¿Cómo protejo y aligero una ruta?	¿Cómo hablan padre e hijo?	¿Cómo comparten datos componentes lejanos?

Pregunta final para todos: ¿qué pantalla de tu proyecto integrador protegerías con un guard y por qué?

5 · CIERRE · EVALUACIÓN · PREGUNTA 1

Pregunta 1

¿En qué lugar dibuja Angular el componente que corresponde a la URL actual?

- **A** Reemplaza todo el contenido de <app-root>
- **B** Dentro de la etiqueta <router-outlet />
- **C** En un archivo HTML nuevo que envía el servidor
- **D** Al final del <body> de index.html

5 · CIERRE · EVALUACIÓN · PREGUNTA 2

Pregunta 2

ProductCard debe avisar a Catalogo que el usuario presionó «Agregar al carrito». ¿Qué debe usar ProductCard?

- **A** Un input() con el producto
- **B** Un viewChild() del catálogo
- **C** Un output() y llamar a emit()
- **D** Un routerLink hacia /carrito

5 · CIERRE · EVALUACIÓN · PREGUNTA 3

Pregunta 3

Las rutas son: [{ path: '', component: NoEncontrado }, { path: 'catalogo', component: Catalogo }]. ¿Qué se muestra al visitar /catalogo?**

- **A** NoEncontrado, porque el Router usa la primera ruta que coincide
- **B** Catalogo, porque es la ruta más específica
- **C** Un error de compilación
- **D** Una pantalla en blanco

5 · CIERRE · FUENTES

Fuentes consultadas

Fuente	Contenido	Enlace
Angular (documentación oficial)	Guía de Routing: rutas, outlets, enlaces, guards y carga diferida	angular.dev/guide/routing
Angular (documentación oficial)	Inputs y outputs de componentes; model()	angular.dev/guide/components/inputs
Angular (documentación oficial)	Signals: signal, computed, update	angular.dev/guide/signals
Angular API	withComponentInputBinding	angular.dev/api/router/withComponentInputBinding
Angular 22 (junio de 2026)	Versión estable vigente usada en el curso	angular.dev
Sílabo 100000ST61 – UTP	Logro de la unidad 3 y tema de la sesión 12	Aula virtual del curso
Cardador Cabello, A. L.	Desarrollo de aplicaciones web distribuidas (UF1846). IC Editorial	tubiblioteca.utp.edu.pe

57

5 · CIERRE · MENSAJE FINAL

¡Gracias por tu participación!

«Hoy tu aplicación aprendió a navegar y a conversar. Cada ruta que configures y cada componente que conectes te acerca al producto que pondrás en tu portafolio. Practica, equivócate en el código y vuelve a intentarlo: así se forma un desarrollador.»

Practica

Completa el Taller 12 y el ejercicio avanzado de favoritos.

Participa

Comparte tu avance o tu duda en el foro del curso.

Te esperamos

Sesión 13: formularios, validación y manejo de errores.

58